
FinPilot

Personal Finance & Investment Dashboard

Full-Stack Development Internship

Internship Project Documentation

Organization: Codveda Technologies

Project: FinPilot – Personal Finance & Investment Dashboard

Project Type: Full-Stack Web Application (MERN)

Submitted by: Sushil Bhatta

Live App: <https://finpilot-oiek-q4zxkmfqt-sushil-bhattas-projects.vercel.app/>

API Base: <https://finplot-rhir.onrender.com>

Repository: <https://github.com/Sushil-Bhatta-sb/finpilot>

Contents

1	Project Overview	1
1.1	Introduction	1
1.2	Purpose	1
1.3	Major Features	1
1.4	Architecture	1
1.5	Technology Stack	2
1.6	Authentication Behavior	2
1.7	Project Structure	2
2	Level 1 — Basic	3
2.1	Task 1 — Development Environment Setup	3
2.1.1	Objective	3
2.1.2	Tools Installed	3
2.1.3	Version Control	3
2.1.4	Verification	3
2.1.5	Contribution to FinPilot	4
2.2	Task 2 — REST API	4
2.2.1	Objective	4
2.2.2	Technologies Used	4
2.2.3	Implementation	4
2.2.4	Error Handling	5
2.2.5	Testing and Verification	5
2.2.6	Contribution to FinPilot	7
3	Level 2 — Intermediate	8
3.1	Task 1 — Frontend Framework	8
3.1.1	Objective	8
3.1.2	Technologies Used	8
3.1.3	Implementation	8
3.1.4	State Management and Feedback	8
3.1.5	Styling	8
3.1.6	Frontend Project Structure	9
3.1.7	Screenshots	9
3.1.8	Contribution to FinPilot	10
3.2	Task 2 — Authentication	10
3.2.1	Objective	10
3.2.2	Technologies Used	11
3.2.3	Auth Endpoints	11
3.2.4	Password Hashing	11
3.2.5	JWT Flow	11
3.2.6	Role-Based Access Control	11
3.2.7	Testing and Verification	11
3.2.8	Contribution to FinPilot	14
3.3	Task 3 — Database Integration	14
3.3.1	Objective	14

3.3.2	Technologies Used	14
3.3.3	Models	15
3.3.4	Relationships	15
3.3.5	Data Validation	15
3.3.6	Indexing and Optimization	15
3.3.7	CRUD Endpoints	16
3.3.8	Connection	16
3.3.9	Testing and Verification	16
3.3.10	Contribution to FinPilot	18
4	Level 3 — Advanced	19
4.1	Task 1 — Full-Stack Application	19
4.1.1	Objective	19
4.1.2	Modules Implemented	19
4.1.3	Role-Based Access	19
4.1.4	UI/UX Design	19
4.1.5	Screenshots	20
4.1.6	Deployment Readiness	23
4.1.7	Contribution to FinPilot	23
4.2	Task 2 — WebSockets	23
4.2.1	Objective	23
4.2.2	Why WebSockets Are Used	23
4.2.3	Technologies Used	23
4.2.4	How the Client Communicates with the Backend	23
4.2.5	How Real-Time Notifications Work	24
4.2.6	Testing and Verification	24
4.2.7	Contribution to FinPilot	26
5	API Reference	27
6	Security	29
7	Deployment	30
8	Final Project Summary	31
8.1	What Was Built	31
8.2	Technologies Learned and Applied	31
8.3	Full-Stack Concepts Demonstrated	31
8.4	Deployment Status	31
8.5	Overall Outcome	31

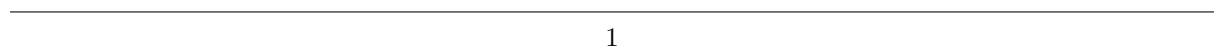
Project Overview

FinPilot is a full-stack personal finance dashboard for tracking income, expenses, budgets, savings goals, investments, and transactions. It is designed for individual users who want a single, privacy-respecting dashboard to monitor their personal finances. The application provides a consolidated view of income and expenses, budget progress and alerts, savings goals, an investments tracker, transaction history with export options, and role-based admin tools.

The goal of FinPilot is to give an individual user one place to understand where their money comes from and where it goes. Every financial record is scoped to the authenticated user, so a user can only view and modify their own data. Admin users additionally receive platform-level management and reporting tools.

- **Authentication** — JWT-based auth with bcrypt password hashing and role-based access control.
- **Financial Tracking** — Create and manage income, expenses, budgets (with exceed alerts), savings goals, and investments.
- **Insights** — Interactive dashboard with charts and category breakdowns.
- **Real-Time** — Socket.io powered per-user notifications and toast alerts.
- **Admin Tools** — User management, suspend accounts, and platform reporting for admins.
- **Data Export** — Export transaction history to CSV and PDF.

FinPilot follows a classic three-tier full-stack architecture. The React frontend communicates with the Express REST API over HTTP, and the API persists data in MongoDB through Mongoose. A Socket.io channel runs alongside the REST API to deliver real-time notifications to individual users.



1.5 Technology Stack

Layer	Technologies
Frontend	React, TypeScript, Vite, React Router, Axios, Recharts, Socket.io Client
Backend	Node.js, Express, JWT, bcryptjs, Socket.io, PDFKit
Database	MongoDB (Mongoose)
Deployment	Vercel (frontend), Render (backend), MongoDB Atlas

Table 1.1: FinPilot technology stack by layer.

1.6 Authentication Behavior

The dashboard and general pages can initially be viewed without logging in — the dashboard statistics endpoint returns a guest-safe payload when the request is unauthenticated. Authentication is required only when accessing personal or database-specific functionality. Protected operations include all user-specific financial data (income, expenses, budgets, savings, investments, transactions, categories) and profile/account operations. These protections are enforced on the backend as well, so the API rejects unauthenticated access to private routes regardless of the client used.

1.7 Project Structure

```

1 finpilot/
2 |- backend/
3 |   |- server.js
4 |   |- render.yaml
5 |   \- src/
6 |       |- app.js
7 |       |- socket.js
8 |       |- config/
9 |       |- controller/
10 |      |- middleware/
11 |      |- models/
12 |      \- routes/
13 |- frontend/
14 |   |- package.json
15 |   \- src/
16 |       |- api/
17 |       |- components/
18 |       |- context/
19 |       |- pages/
20 |       \- types/
21 \- docs/
22     |- level-1/
23     |- level2/
24     \- level-3/

```

Listing 1.1: FinPilot repository structure.

CHAPTER 2

Level 1 — Basic

2.1 Task 1 — Development Environment Setup

2.1.1 Objective

Set up and verify a working local development environment for building the FinPilot full-stack application, and initialize version control.

2.1.2 Tools Installed

Tool	Version / Detail
Node.js	v20.x
npm	v10.x
Git	v2.x
Code Editor	VS Code
Database	MongoDB Atlas (cloud-hosted)

Table 2.1: Development environment toolchain.

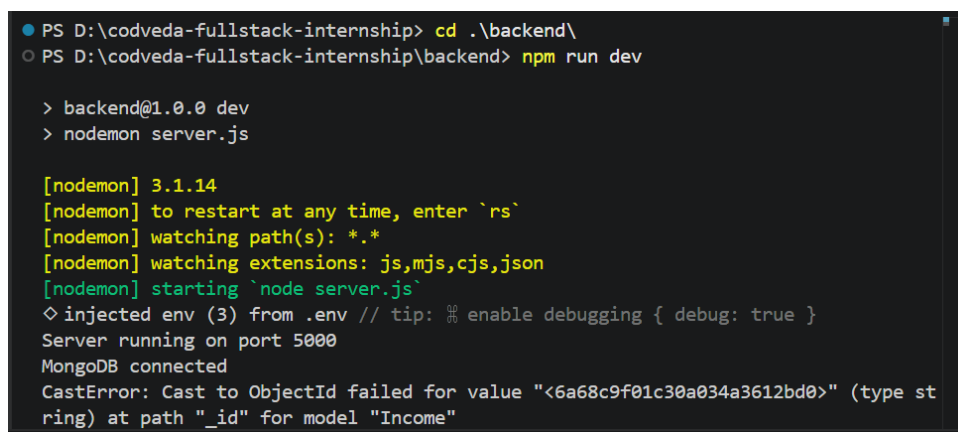
2.1.3 Version Control

The project repository is hosted on GitHub at <https://github.com/Sushil-Bhatta-sb/finpilot>. Basic Git commands were practiced as part of the setup: `git init`, `git add .`, `git commit -m "message"`, and `git push origin main`.

2.1.4 Verification

```
PS D:\codveda-fullstack-internship> node -v
>> npm -v
● >> git --version
v22.15.0
10.9.2
git version 2.51.0.windows.1
PS D:\codveda-fullstack-internship>
```

Figure 2.1: Verifying installed Node.js, npm, and Git versions.



```

PS D:\codveda-fullstack-internship> cd .\backend\
PS D:\codveda-fullstack-internship\backend> npm run dev

> backend@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (3) from .env // tip: ⌘ enable debugging { debug: true }
Server running on port 5000
MongoDB connected
CastError: Cast to ObjectId failed for value "<6a68c9f01c30a034a3612bd0>" (type st
ring) at path "_id" for model "Income"

```

Figure 2.2: Successful connection to the MongoDB Atlas database.

2.1.5 Contribution to FinPilot

This task established the toolchain, cloud database, and version control that every later task builds upon, ensuring a reproducible environment for developing and deploying the application.

2.2 Task 2 — REST API

2.2.1 Objective

Build a simple REST API providing full CRUD operations for two core resources of the finance domain: **Income** and **Expense**.

2.2.2 Technologies Used

Node.js, Express, and MongoDB via the Mongoose ODM.

2.2.3 Implementation

A REST API was implemented with full CRUD for the Income and Expense resources, input validation, and centralized error handling. The endpoints follow REST conventions and return JSON responses.

Method	Endpoint	Description
GET	/api/income	Get all income
POST	/api/income	Create income entry
PUT	/api/income/:id	Update income entry
DELETE	/api/income/:id	Delete income entry
GET	/api/expenses	Get all expenses
POST	/api/expenses	Create expense entry
PUT	/api/expenses/:id	Update expense entry
DELETE	/api/expenses/:id	Delete expense entry

Table 2.2: Level 1 REST API endpoints for Income and Expense.

2.2.4 Error Handling

A centralized error-handler middleware returns JSON errors with proper HTTP status codes (404 for not found, 500 for server errors).

2.2.5 Testing and Verification

The API was tested during development using **Thunder Client** (a VS Code extension), and a shareable **Postman** collection was created for submission covering every endpoint (GET, POST, PUT, DELETE for both resources, including error cases). The collection is included at `postman/finpilot-level1.postman_collection.json`; the `baseUrl` variable is set to `http://localhost:5000` and requests are run in order (Create → Get → Update → Delete).

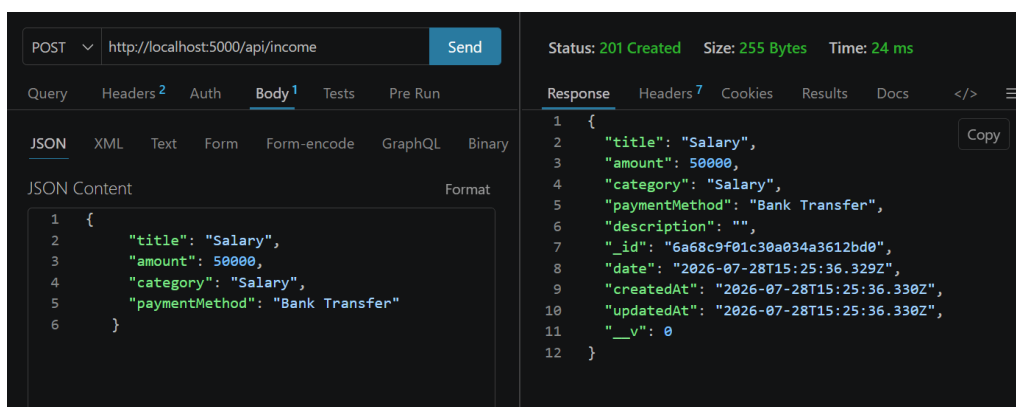


Figure 2.3: Creating an income entry via the REST API.

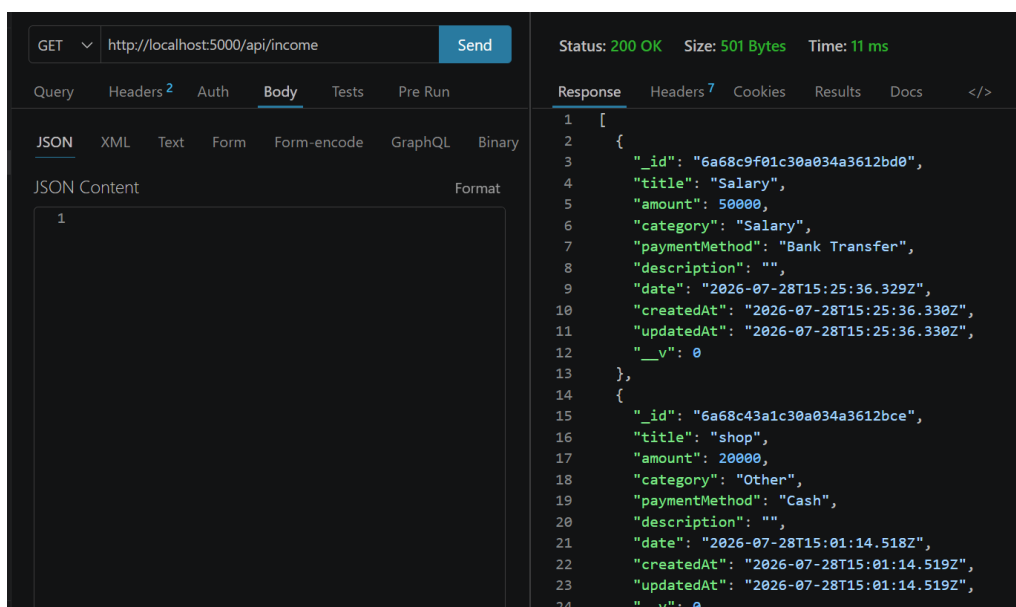


Figure 2.4: Retrieving all income entries.

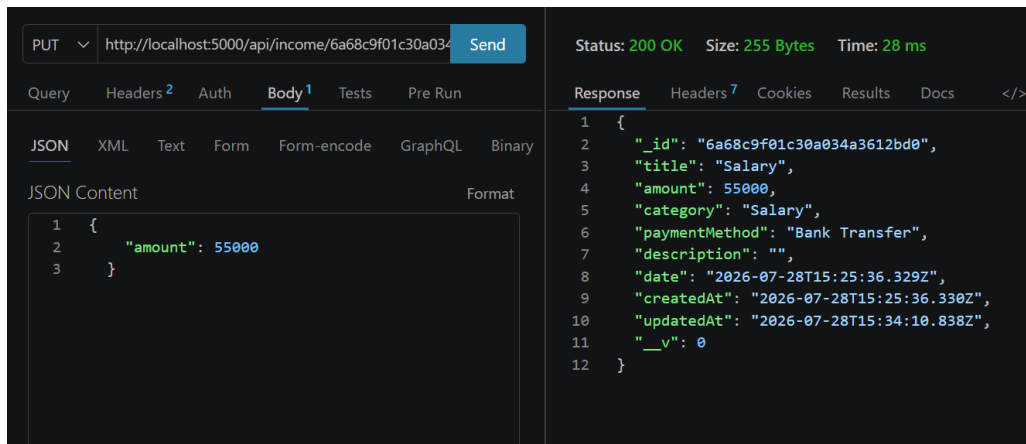


Figure 2.5: Updating an existing income entry.

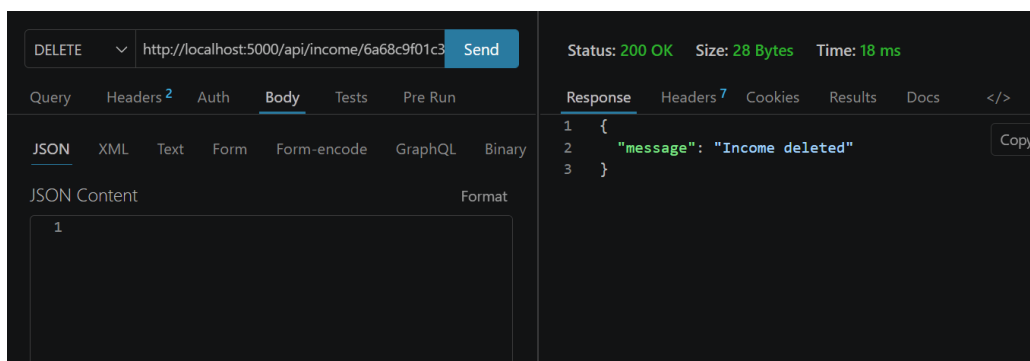


Figure 2.6: Deleting an income entry.

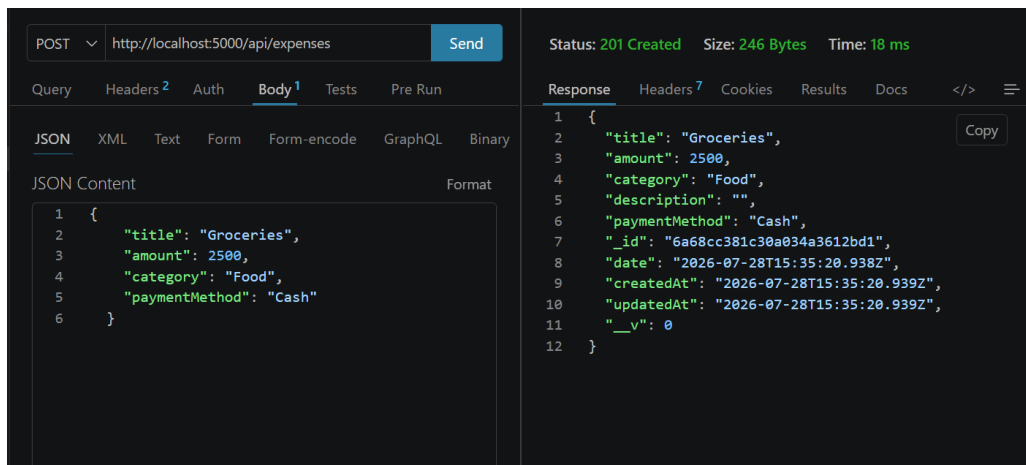


Figure 2.7: Creating an expense entry via the REST API.

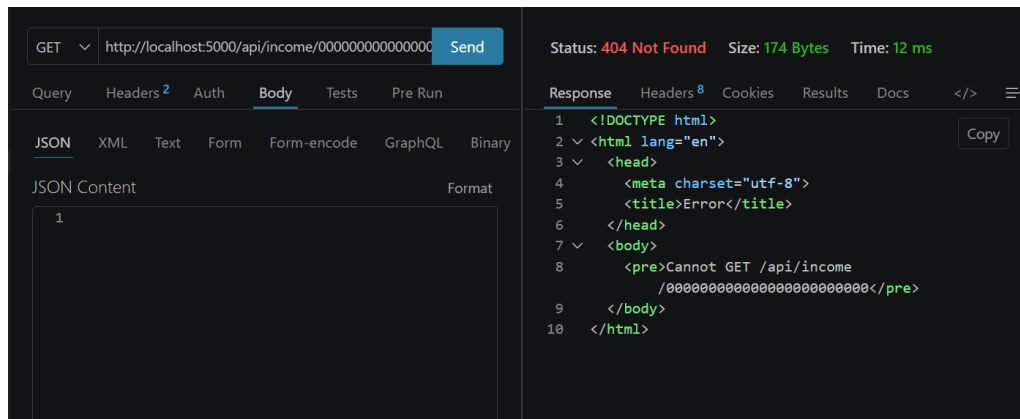


Figure 2.8: Error handling for an invalid resource ID.

2.2.6 Contribution to FinPilot

This task delivered the first working slice of FinPilot's backend — the Income and Expense data model and CRUD endpoints — which the frontend, database integration, and later financial modules extend and build upon.

CHAPTER 3

Level 2 — Intermediate

3.1 Task 1 — Frontend Framework

3.1.1 Objective

Rebuild the FinPilot frontend using a modern JavaScript framework with a component-based, type-safe architecture, centralized state management, and typed API calls with loading states.

3.1.2 Technologies Used

- **React 19 + TypeScript** — component-based, type-safe UI.
- **Vite** — fast dev server and build tool.
- **React Router v7** — client-side routing and protected routes.
- **Axios** — HTTP client with request/response interceptors.

3.1.3 Implementation

The frontend was built as a Vite + React + TypeScript project under `frontend/`. All components use hooks; global authentication state is held in `AuthContext` via the Context API. Typed Axios services live in `src/api/`, and every form and list shows spinners while requests are pending. A small set of reusable UI components was created.

Component	Purpose
Button	Variants (primary/secondary/danger/ghost), built-in loading spinner, full-width option
Input	Labeled input with consistent focus styling
Card	Surface container used for panels and auth forms
Spinner	Inline loading indicator for lists

Table 3.1: Reusable UI components in `src/components/ui/`.

3.1.4 State Management and Feedback

`AuthContext` centralizes the authenticated user, token, and the `login / register / logout` actions. The Dashboard lifts income and expense totals into local state to render a live **Total Income / Total Expenses / Balance** summary. Login and Register buttons show a spinner and disable while a request is in flight; income and expense lists show a loading spinner on first fetch, an empty state when there is no data, and inline error messages on failure.

3.1.5 Styling

A custom design system was added via CSS variables in `frontend/src/index.css` with component styles in `frontend/src/App.css`, including automatic light/dark mode support.

3.1.6 Frontend Project Structure

```
1 frontend/src/  
2   api/           # Axios client + typed service calls (auth, income, expense  
3   )  
4   components/  
5     ui/          # Reusable UI: Button, Input, Card, Spinner  
6     incomeList.tsx  
7     expenseList.tsx  
8     ProtectedRoute.tsx  
9   context/  
10    AuthContext.tsx # Global auth state (login, register, logout)  
11  pages/  
12    Login.tsx  
13    Register.tsx  
14  types/          # Shared TypeScript interfaces  
15  App.tsx          # Routes + Dashboard
```

Listing 3.1: Frontend source layout.

3.1.7 Screenshots

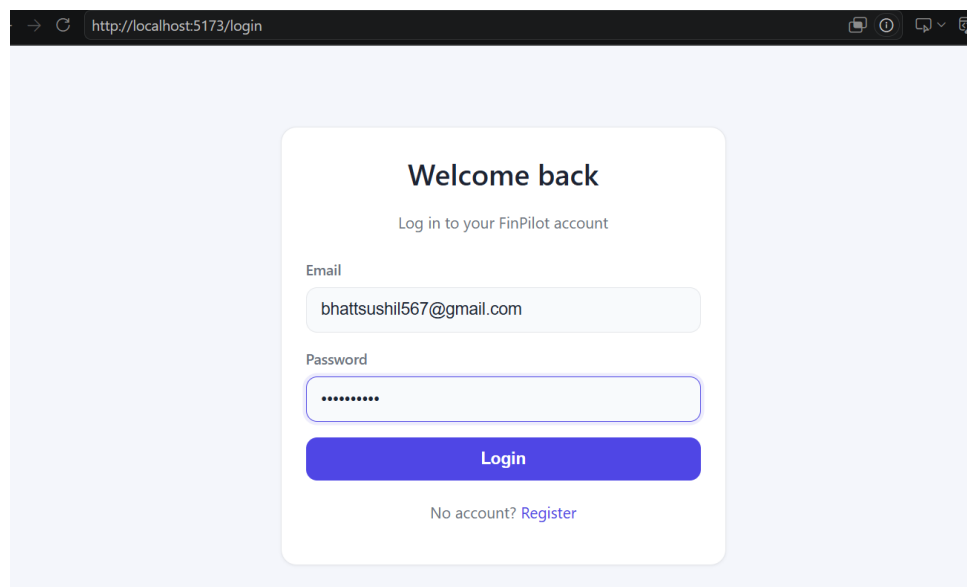


Figure 3.1: Login page built with React and TypeScript.

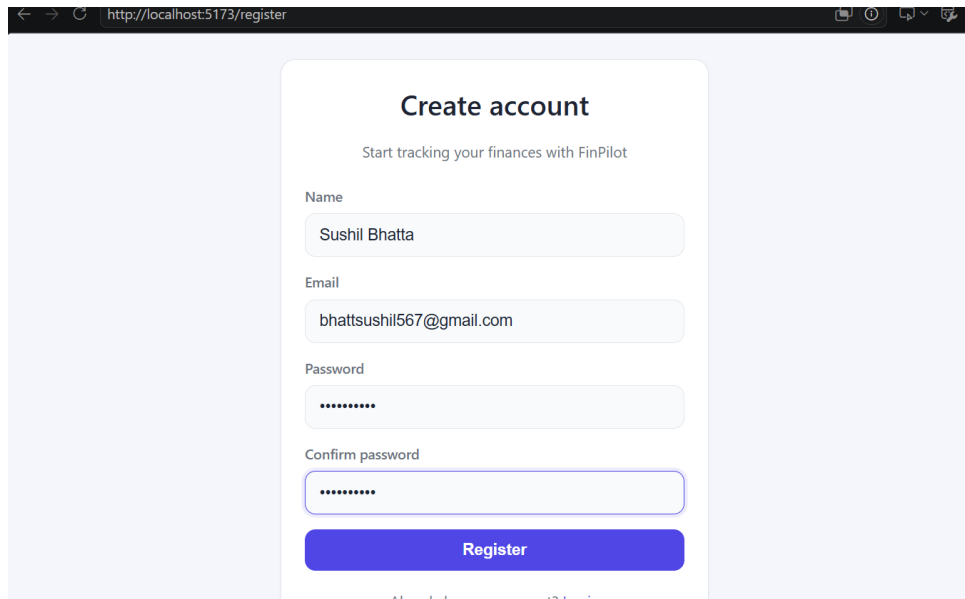


Figure 3.2: Registration page with client-side validation.

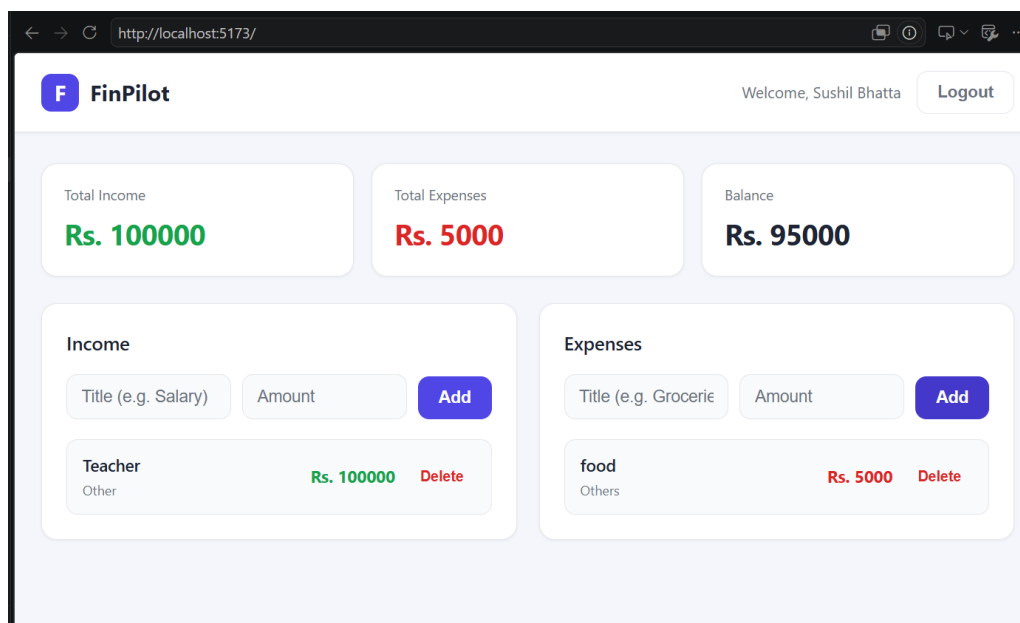


Figure 3.3: Dashboard showing the live income/expense/balance summary.

3.1.8 Contribution to FinPilot

This task delivered the application's user interface foundation — routing, global auth state, typed API clients, and the reusable component library — that all later feature pages (budgets, savings, investments, analytics, admin) reuse.

3.2 Task 2 — Authentication

3.2.1 Objective

Implement user authentication (signup/login) with secure password storage and stateless sessions, protect backend routes, and restrict access based on user roles (`user` / `admin`).

3.2.2 Technologies Used

`bcryptjs` for password hashing and `jsonwebtoken` (JWT) for stateless sessions, integrated on the frontend through an Axios interceptor and `AuthContext`.

3.2.3 Auth Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Create account, returns JWT
POST	/api/auth/login	Public	Login, returns JWT
GET	/api/auth/me	Authenticated	Get current user
GET	/api/auth/users	Admin only	List all users (role-based)

Table 3.2: Authentication endpoints and required access level.

3.2.4 Password Hashing

Passwords are never stored in plain text. The `User` model hashes the password before saving and exposes a `comparePassword()` helper for login.

```
1 userSchema.pre('save', async function () {
2   if (!this.isModified('password')) return;
3   const salt = await bcrypt.genSalt(10);
4   this.password = await bcrypt.hash(this.password, salt);
5 });
```

Listing 3.2: Password hashing in a pre-save hook on the `User` model.

3.2.5 JWT Flow

1. On register/login the server signs a JWT containing the user `id` and `role` (expires in 7 days).
2. The frontend stores the token and attaches it as `Authorization: Bearer <token>` on every request via an Axios interceptor.
3. The `protect` middleware verifies the token, loads the user, and attaches it to `req.user`.
4. On a 401 response the frontend clears the token and redirects to `/login`.

3.2.6 Role-Based Access Control

The `User` model has a `role` field (`enum: ['user', 'admin'], default user`). The `authorize(...roles)` middleware returns 403 `Forbidden` when the user's role is not permitted — for example, `GET /api/auth/users` is guarded by `protect, authorize('admin')`. On the frontend, `AuthContext` verifies the session on load via `GET /api/auth/me` and `ProtectedRoute` redirects unauthenticated users to `/login`.

3.2.7 Testing and Verification

A Postman collection at `postman/finpilot-level2.postman_collection.json` covers register, login, `me`, protected access, and the admin-only route. The flow verifies that a normal user token receives 403 on the admin route while an admin token receives the user list. Error

responses are: 400 (email already registered / invalid input), 401 (missing, invalid, or expired token; wrong credentials), 403 (authenticated but insufficient role), and 404 (user not found).

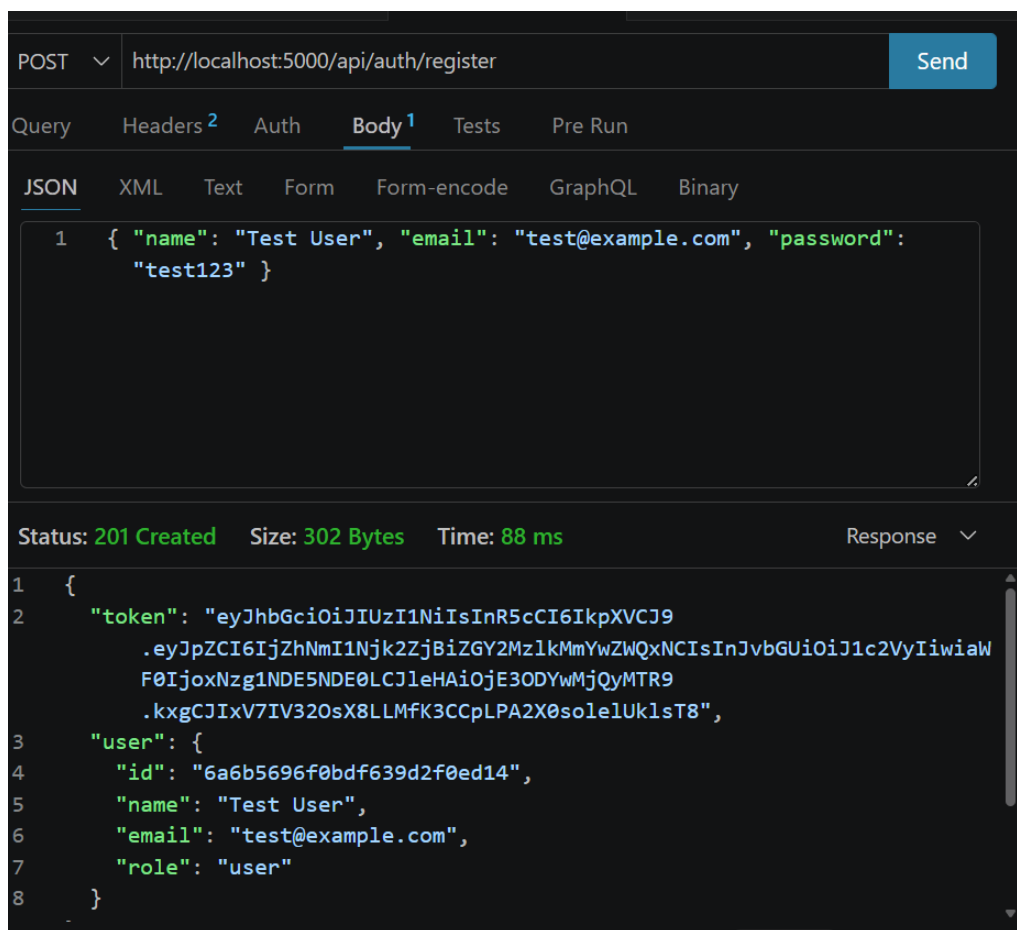


Figure 3.4: Successful registration returning a JWT.

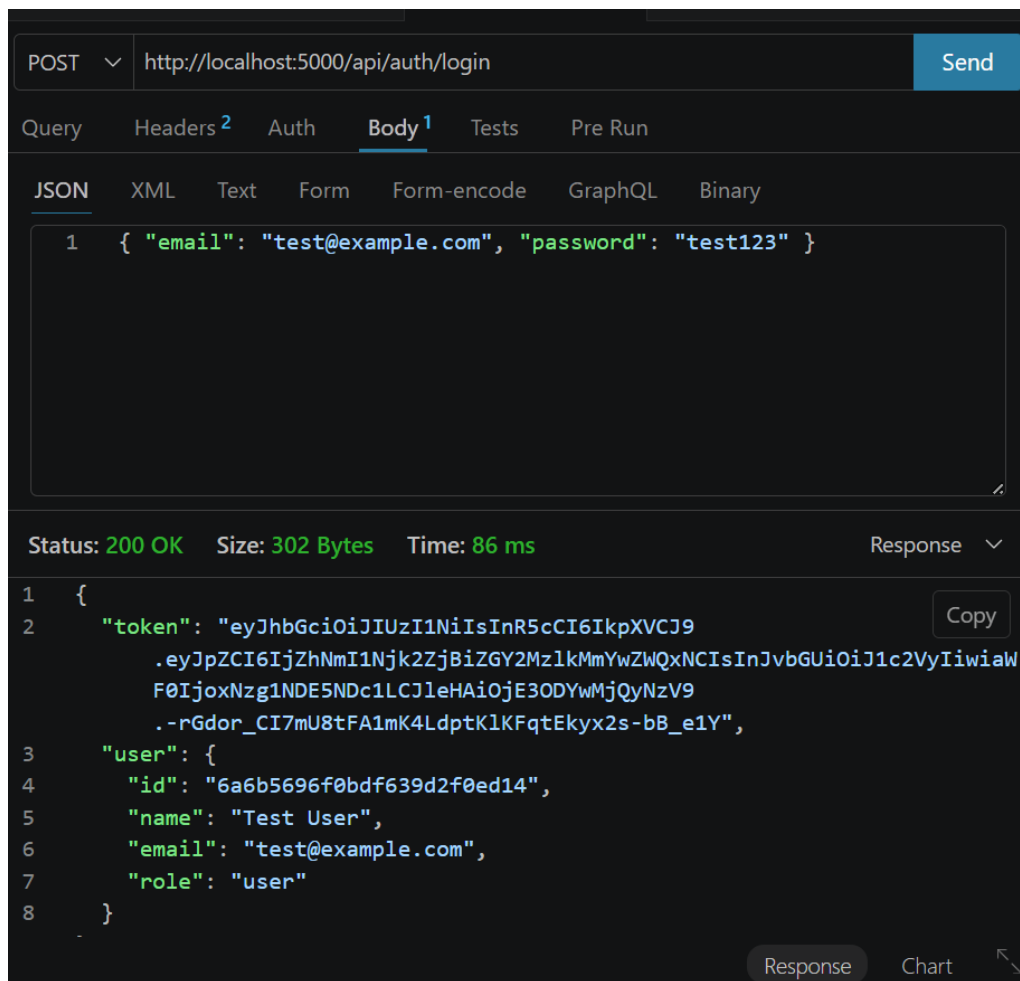


Figure 3.5: Successful login returning a JWT.

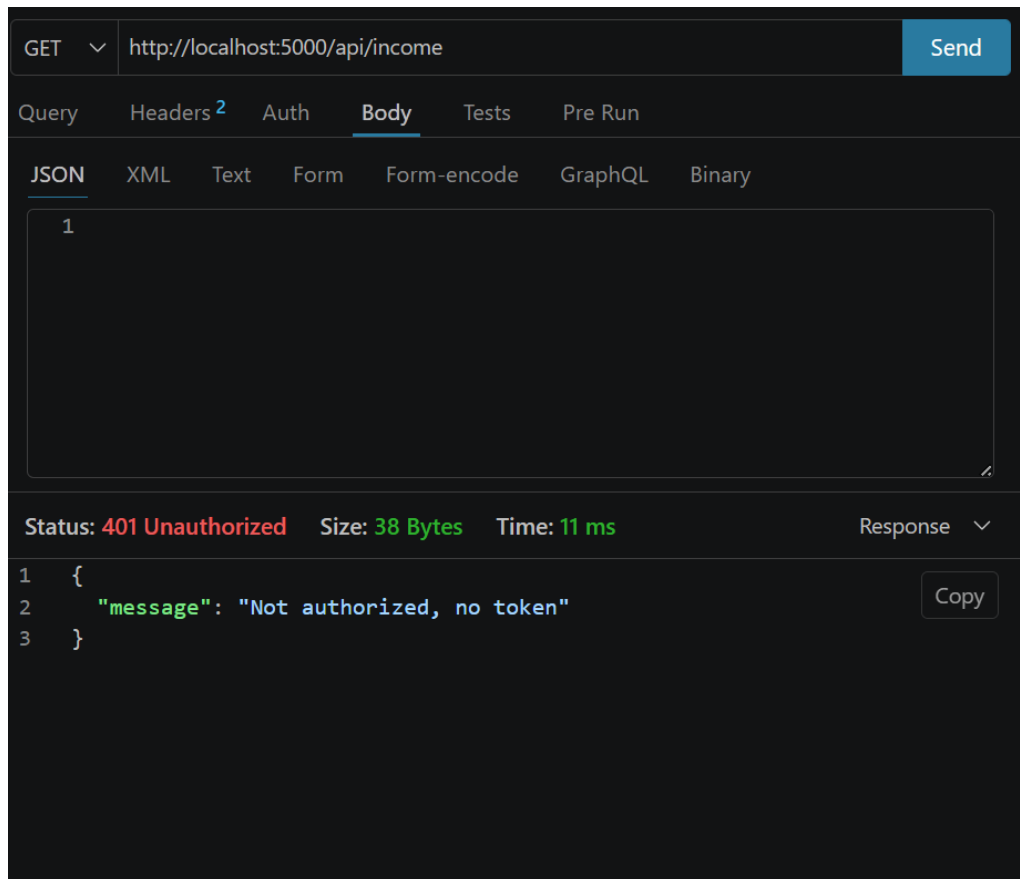


Figure 3.6: Protected route rejecting a request with no token (401).

3.2.8 Contribution to FinPilot

Authentication is the security backbone of FinPilot: it guarantees that every private financial record is tied to and accessible only by its owner, and it gates the admin tools behind role checks.

3.3 Task 3 — Database Integration

3.3.1 Objective

Integrate MongoDB using the Mongoose ODM, define models and relationships, enforce data validation before saving, and add indexes for query optimization, with all CRUD operations scoped to the authenticated user.

3.3.2 Technologies Used

MongoDB (Atlas) with the Mongoose ODM. The connection is defined in `src/config/db.js` and schemas live in `src/models/`.

3.3.3 Models

Model	Key Fields	Notes
User	name, email (unique, lowercase), password (min 6), role (enum user/admin)	Password hashed pre-save
Income	user (ref User), title, amount (min 0), date, category, paymentMethod, description	Indexed by { user, date }
Expense	user (ref User), title, amount (min 0), category, date, paymentMethod, description	Indexed by { user, date }

Table 3.3: Core Mongoose models and their key fields.

3.3.4 Relationships

Income and **Expense** each hold a `user` field that references the **User** collection via a Mongoose `ObjectId` ref. Every controller query filters by `req.user.id`, so a user can only read or modify their own records.

```
1 user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
```

Listing 3.3: User reference on the Income/Expense schema.

3.3.5 Data Validation

Validation runs automatically before documents are saved:

- **Required fields:** `user`, `title`, `amount`.
- **amount:** Number with `min: 0` — negative values are rejected.
- **String fields:** `trim: true` removes accidental whitespace.
- **email:** `unique + lowercase` to prevent duplicate accounts.
- **role:** restricted to `enum: ['user', 'admin']`.
- **password:** `minlength: 6`.

3.3.6 Indexing and Optimization

A compound index { `user: 1, date: -1` } on both **Income** and **Expense** makes the common "list my records, newest first" query fast. The `unique` constraint on `email` creates an index that enforces one account per email and speeds up login lookups.

3.3.7 CRUD Endpoints

Method	Endpoint	Description
GET	/api/income	List current user income
POST	/api/income	Create income entry
PUT	/api/income/:id	Update income entry
DELETE	/api/income/:id	Delete income entry
GET	/api/expenses	List current user expenses
POST	/api/expenses	Create expense entry
PUT	/api/expenses/:id	Update expense entry
DELETE	/api/expenses/:id	Delete expense entry

Table 3.4: User-scoped CRUD endpoints backed by MongoDB.

3.3.8 Connection

```
1 mongoose.connect(process.env.MONGO_URI)
```

Listing 3.4: MongoDB connection via Mongoose.

The URI is stored in `backend/.env` and is never committed.

3.3.9 Testing and Verification

The centralized error handler returns JSON with proper status codes: 400 for Mongoose validation errors (missing/invalid fields, negative amount), 403 when attempting to modify a record owned by another user, and 500 for unexpected server/database errors.

```

$ npm run dev
◇ injected env (3) from .env // tip: ⓘ custom filepath { path: '/c
ustom/path/.env' }
Server running on port 5000
MongoDB connected
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (3) from .env // tip: ⓘ suppress logs { quiet: true
}
Server running on port 5000
MongoDB connected
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (3) from .env // tip: ⓘ custom filepath { path: '/c
ustom/path/.env' }
Server running on port 5000
MongoDB connected
ValidationError: Income validation failed: amount: Path `amount` i

```

Figure 3.7: MongoDB Atlas connected to the application.

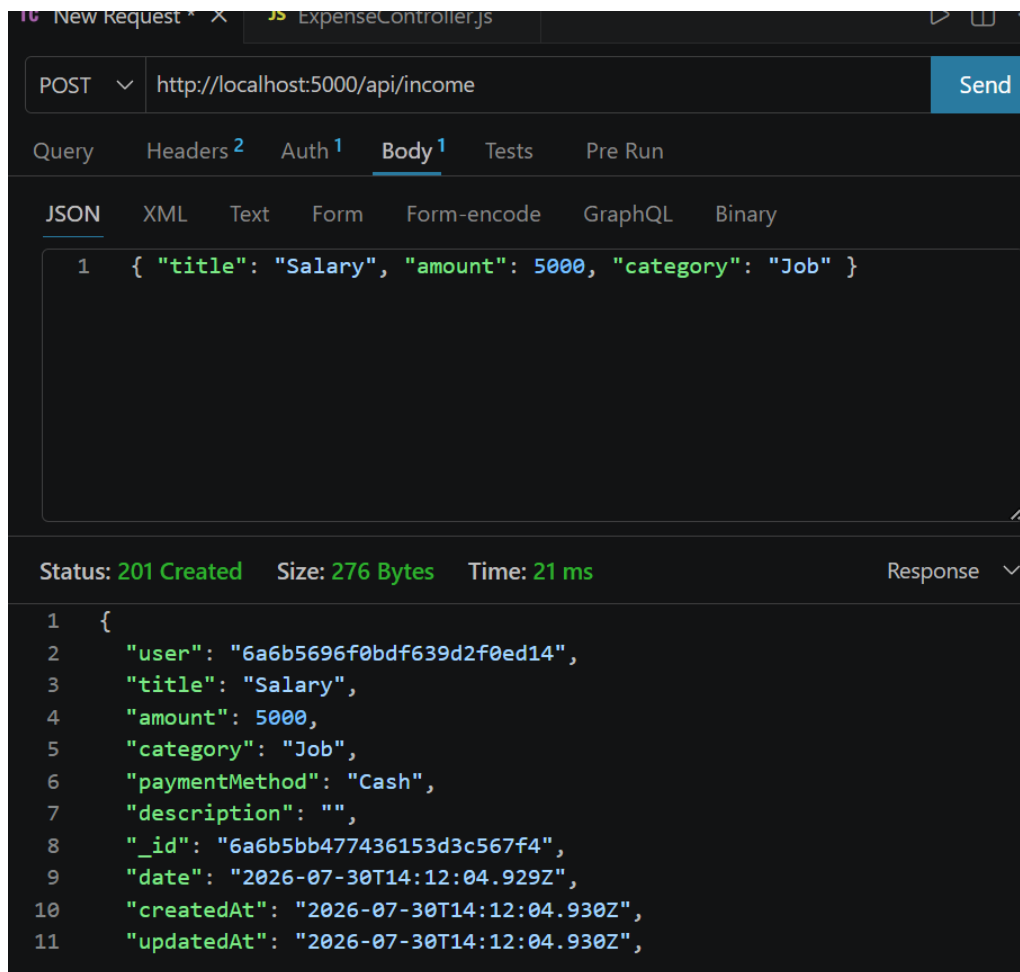


Figure 3.8: Creating a record that passes schema validation.

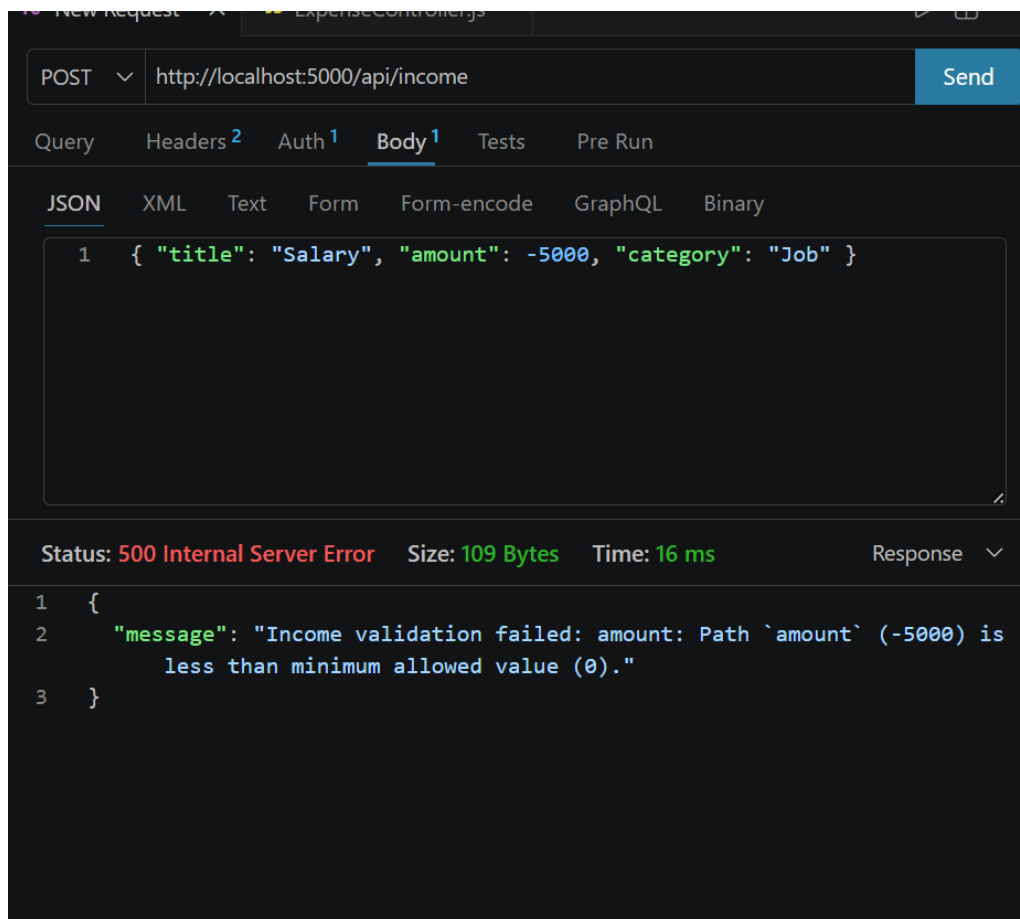


Figure 3.9: A negative amount rejected by validation.

3.3.10 Contribution to FinPilot

Database integration turns FinPilot's in-memory operations into durable, per-user persisted data with validation and fast queries — the storage foundation every financial module depends on.

CHAPTER 4

Level 3 — Advanced

4.1 Task 1 — Full-Stack Application

4.1.1 Objective

Extend FinPilot into a complete full-stack (MERN) personal finance application covering all core modules with full role-based access control (user/admin).

4.1.2 Modules Implemented

Module	Description
Authentication	Register, login, JWT, bcrypt password hashing, protected routes
Dashboard	Aggregated stats, charts (income vs expense, category breakdown, budget usage)
Income & Expense	Full CRUD, scoped per user
Budgets	Create/edit/delete, auto-calculated spent amount, exceed alerts
Savings Goals	Create goals, deposit money, auto-complete on target reached
Investments	Track stocks/crypto/gold/etc., profit/loss calculation
Transactions	Unified activity log, search/filter/sort, CSV & PDF export
Categories	Custom category CRUD per user
Analytics	Charts for income growth, savings growth, category spending
Profile	Update personal info, change password, theme preference
Admin Panel	User management, suspend users, view platform-wide stats

Table 4.1: Full-stack modules implemented in Level 3.

4.1.3 Role-Based Access

Regular users can only access and modify their own data, enforced via `user` ownership checks on every query. Admin-only routes are protected by the `authorize('admin')` middleware, covering user management and platform statistics. (The underlying JWT/bcrypt mechanism is described in Chapter 3, Task 2.)

4.1.4 UI/UX Design

The application uses a dashboard-style layout with a collapsible, grouped sidebar navigation instead of a flat link list, to keep the interface manageable as the feature count grew.

- **Grouped sections:** Overview, Money (Income/Expenses/Budgets), Growth (Savings/Investments), Records (Transactions/Categories), Insights (Analytics), and a pinned Account section (Profile/Admin/Logout).

- Sidebar collapses to an icon-only rail with flyout popovers, expandable on demand; the active route auto-expands its parent group and is highlighted.
- Fully responsive: overlay drawer navigation on mobile.
- Reusable Modal component for create/edit forms (Budgets, Investments, Categories, Savings deposits) instead of always-visible inline forms.
- Toast notification system for form feedback and real-time Socket.io alerts.
- Skeleton loading states and empty-state messaging on all list views.
- Light/dark theme support via CSS custom properties, respected across every component.
- Consistent Card and Badge components for stats, budget progress, and status indicators.

4.1.5 Screenshots

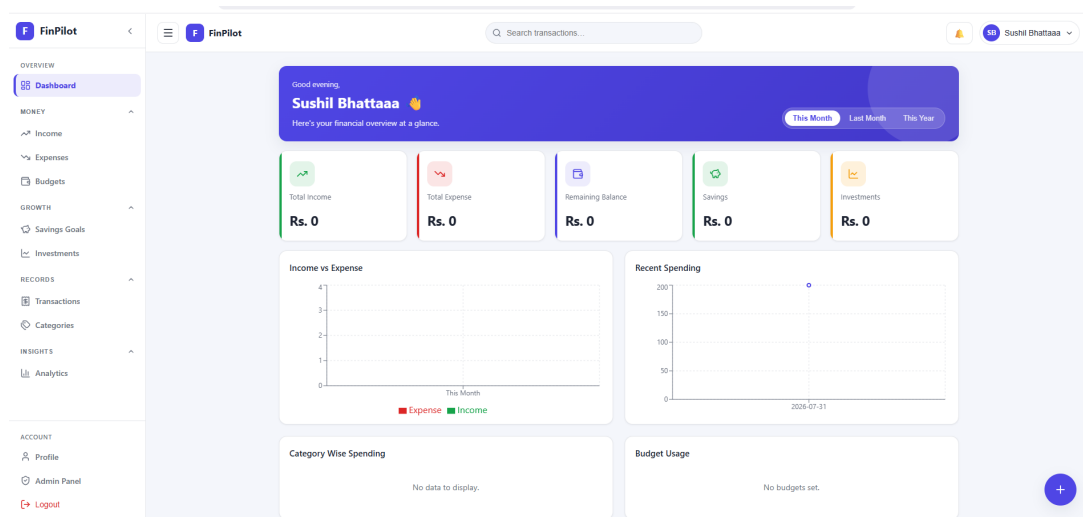


Figure 4.1: Dashboard overview with charts and aggregated stats.

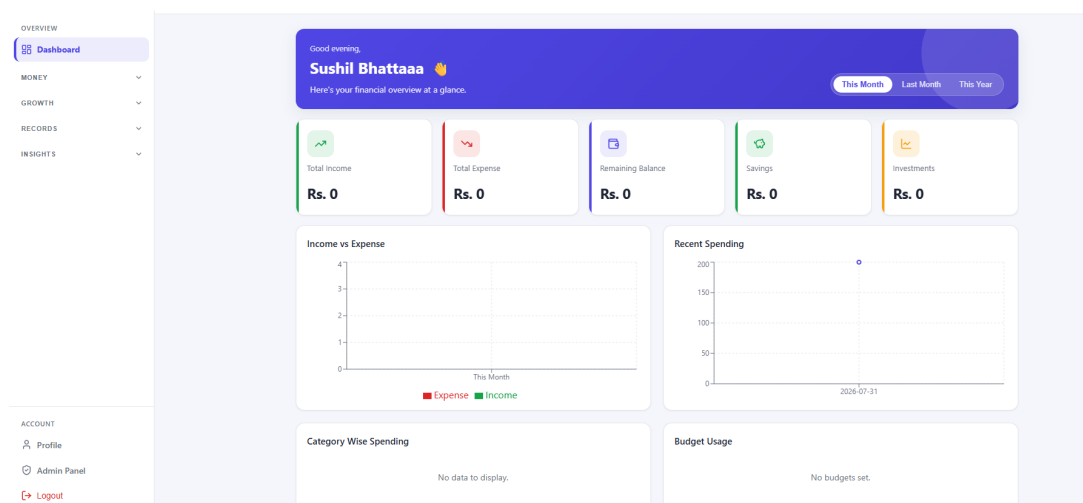


Figure 4.2: Collapsed icon-only sidebar navigation.

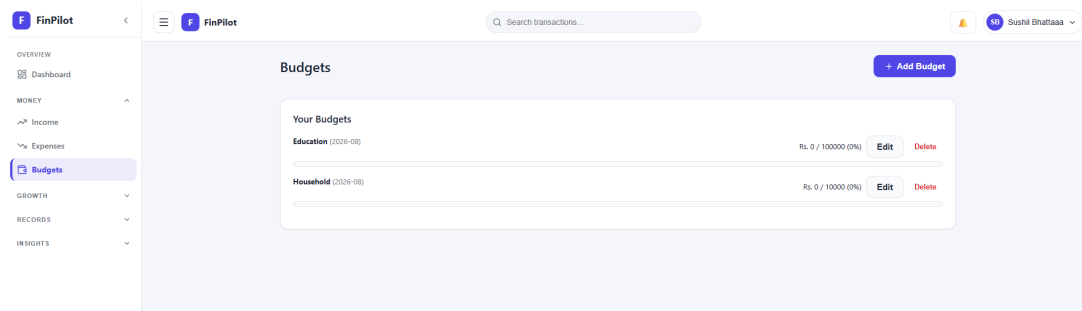


Figure 4.3: Budgets with progress bars and exceed alerts.

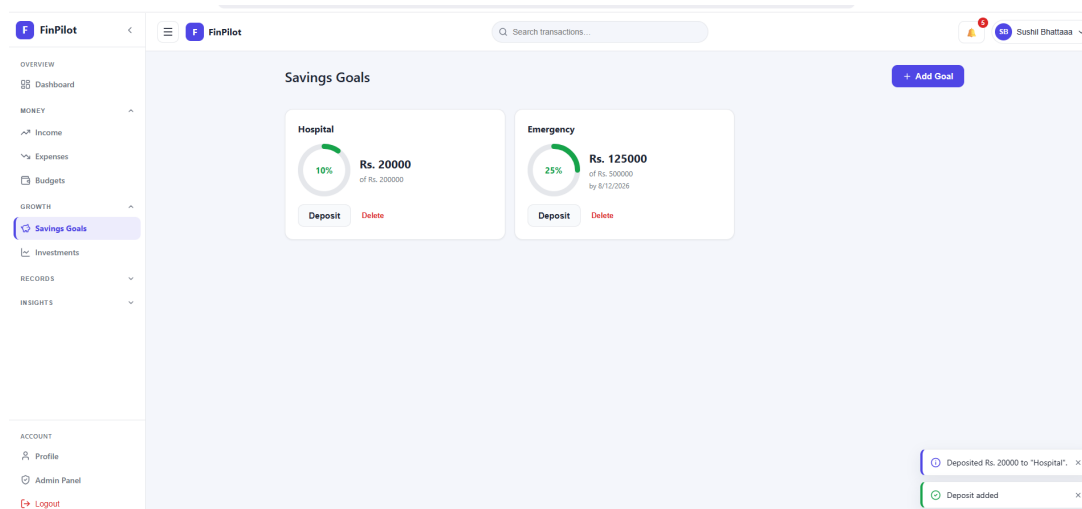


Figure 4.4: Savings goals with goal tracking and deposits.

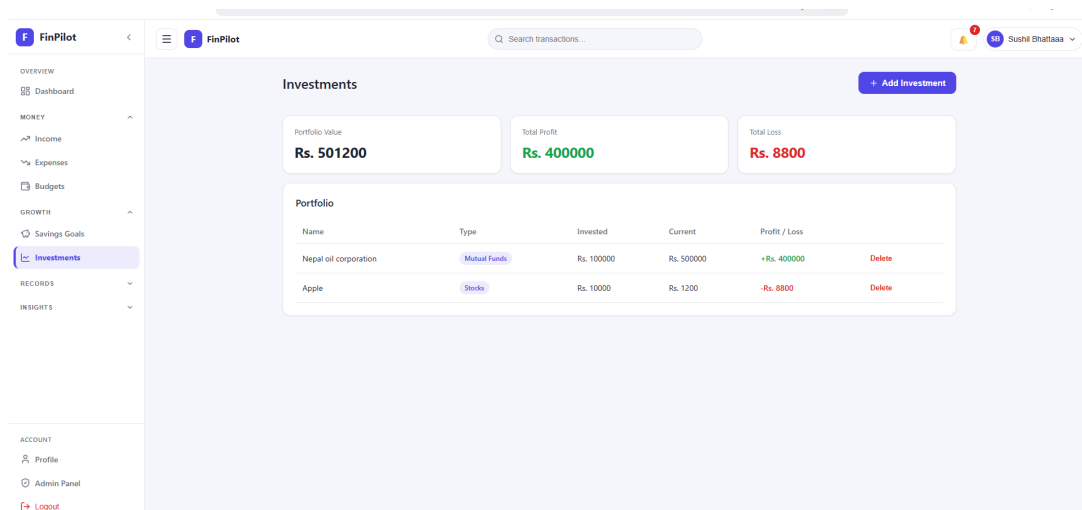


Figure 4.5: Investments portfolio with profit/loss.

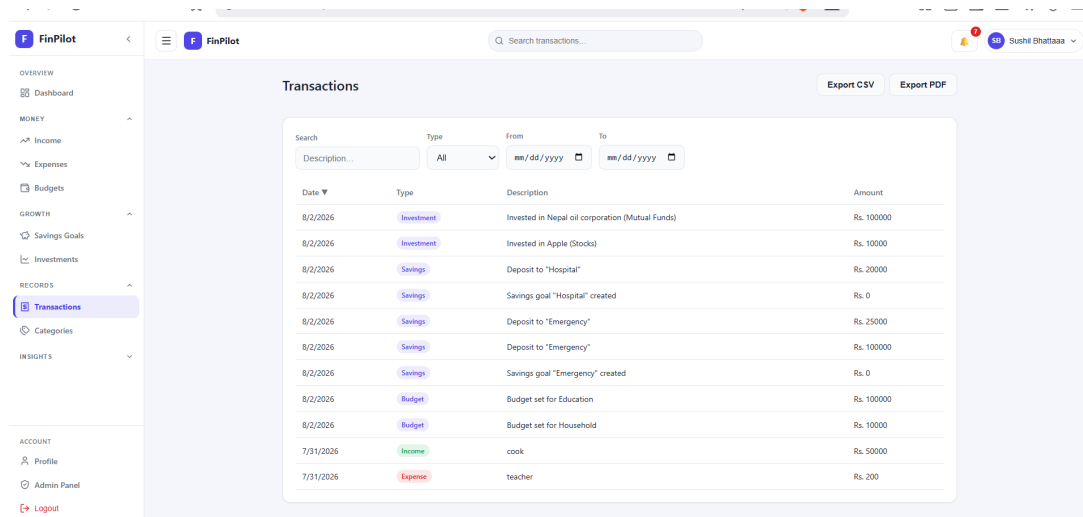


Figure 4.6: Transactions with search, filter, and export.

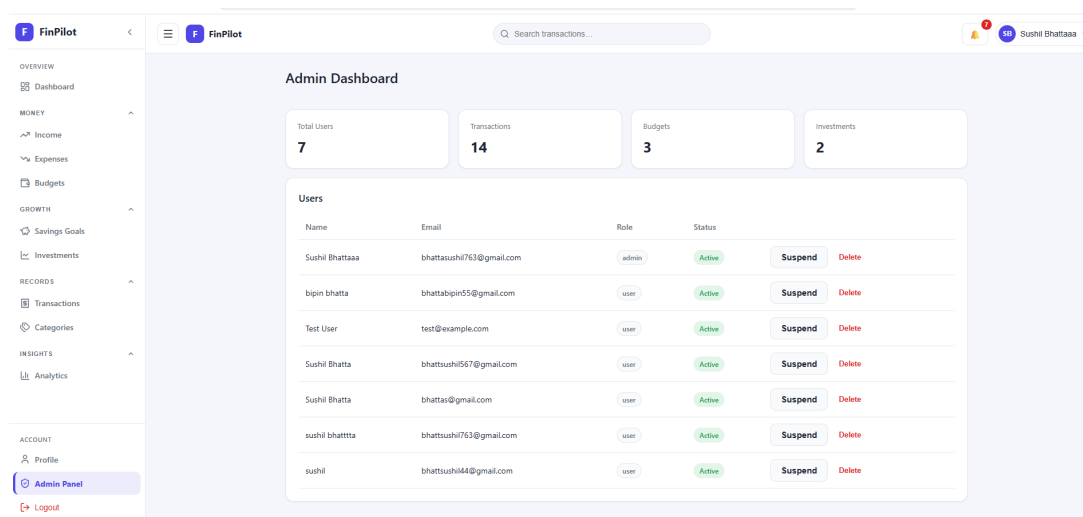


Figure 4.7: Admin panel with user management and platform stats.

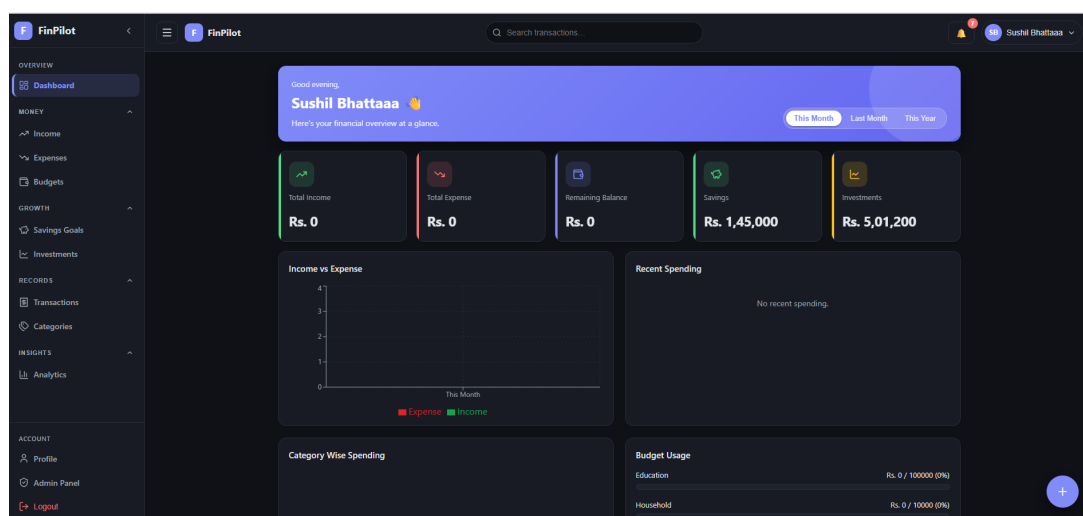


Figure 4.8: Light/dark mode support across the application.

4.1.6 Deployment Readiness

The backend is configured for Render (env vars: MONGO_URI, JWT_SECRET, FRONTEND_URL, PORT); the frontend is configured for Vercel with SPA rewrite rules; and the database uses MongoDB Atlas (cloud-hosted, in use since Level 1).

4.1.7 Contribution to FinPilot

This task integrates every prior building block — auth, UI framework, and database — into the complete FinPilot product, adding the budgets, savings, investments, transactions, analytics, and admin modules that define the app.

4.2 Task 2 — WebSockets

4.2.1 Objective

Deliver private, real-time notifications to users using WebSockets so that finance events (such as budget alerts and savings-goal completion) reach the user instantly without a page refresh.

4.2.2 Why WebSockets Are Used

FinPilot generates events that should reach the user immediately — for example, a budget being exceeded or a savings goal being completed. Polling the REST API for these would be wasteful and slow, so a persistent Socket.io channel is used to push notifications to the specific user the moment they occur.

4.2.3 Technologies Used

Socket.io on the backend (attached to the same HTTP server as the REST API) and the Socket.io client on the React frontend.

4.2.4 How the Client Communicates with the Backend

Socket.io is attached to the HTTP server at startup, with CORS restricted to the frontend origin. Each client, after authenticating, emits a `join` event with its user ID to join a private room named after that user ID. The backend then emits notification events only to that room, keeping every user's notifications private.

```
1 io.on('connection', (socket) => {
2   // Client emits 'join' with their userId; join a private room for that user
3   .
4   socket.on('join', (userId) => {
5     if (userId) socket.join(String(userId));
6   });
7 }
8 // Safe emit helper: no-ops if io is not initialized (e.g. during tests).
9 function emitToUser(userId, event, payload) {
10   if (!io) return;
11   io.to(String(userId)).emit(event, payload);
12 }
```

Listing 4.1: Backend: a per-user private room on connection (socket.js).

4.2.5 How Real-Time Notifications Work

When the backend needs to notify a user, it persists a notification and pushes it to that user's room in a single helper. The client listens for the `notification` event and updates the notification bell and toast alerts in real time.

```
1 async function notify(userId, message, type) {
2   const notification = await Notification.create({ user: userId, message,
3     type });
4   emitToUser(userId, 'notification', notification);
5   return notification;
6 }
```

Listing 4.2: Backend: persist and push a notification (`notify.js`).

```
1 const socket = io(SOCKET_URL, { transports: ['websocket', 'polling'] });
2 socket.emit('join', user.id);
3 socket.on('notification', (payload) => {
4   setNotifications((prev) => [payload, ...prev]);
5   showToast(payload.message, /* type derived from payload.type */);
6 });
7 // ...
8 return () => { socket.disconnect(); };
```

Listing 4.3: Frontend: join the room and react to notification events (`SocketContext`).

On the client, the toast severity is derived from the notification type (`budget-alert` → warning, `goal-completed` → success, otherwise info). The socket connects only when the user is authenticated and disconnects on logout.

4.2.6 Testing and Verification

The implementation was verified to confirm that:

- Notifications are delivered instantly without a page refresh.
- Notifications are private — only the relevant user's room receives their own events.
- The socket disconnects cleanly on logout.

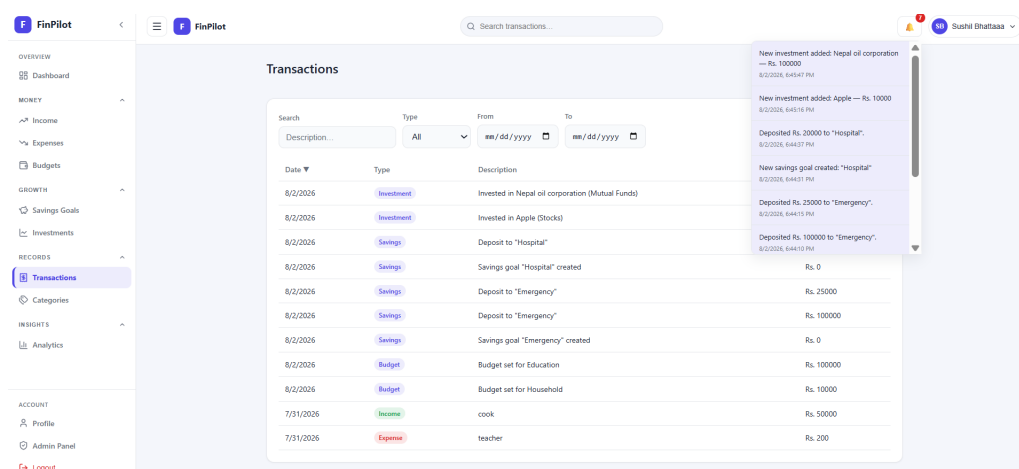


Figure 4.9: Live notification toast delivered in real time.

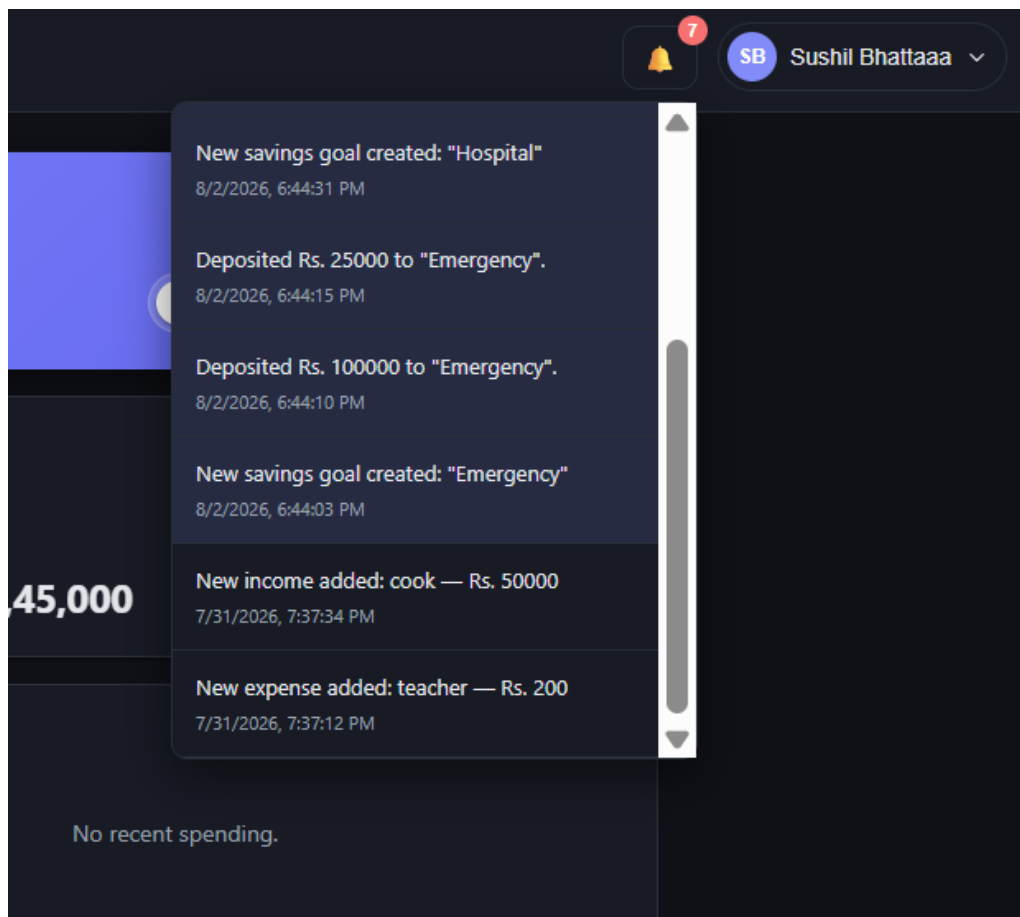


Figure 4.10: Notification bell dropdown listing user notifications.

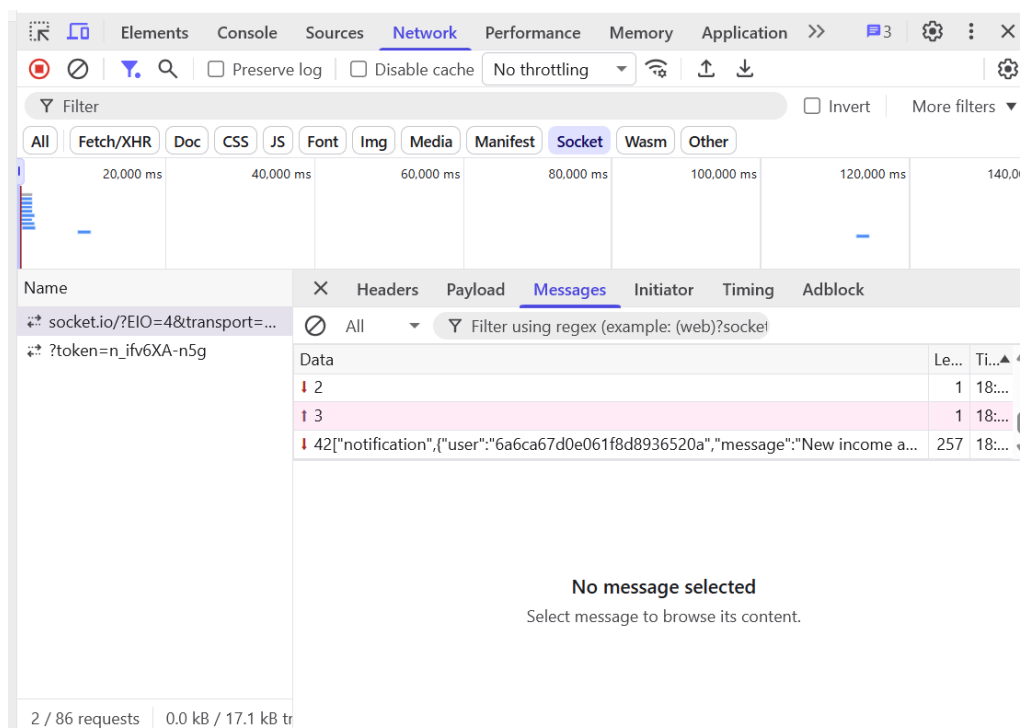


Figure 4.11: Active Socket.io connection shown in browser devtools.

4.2.7 Contribution to FinPilot

WebSockets make FinPilot feel live: budget alerts, savings-goal completion, and other events surface to the correct user immediately, turning the finance data into an interactive, responsive experience.

CHAPTER 5

API Reference

The complete set of REST endpoints exposed by the FinPilot backend is summarized below. Endpoints marked *Required* need a valid JWT; *Admin* endpoints additionally require the admin role; *Public* endpoints are accessible without authentication.

Method	Endpoint	Description	Auth
POST	/api/auth/register	Create account, returns token and user	Public
POST	/api/auth/login	Login, returns token and user	Public
GET	/api/auth/me	Get current authenticated user	Required
GET	/api/auth/users	List users (admin only)	Admin
GET/POST	/api/income	List / create incomes (scoped to user)	Required
PUT/DELETE	/api/income/:id	Update / delete an income owned by the user	Required
GET/POST	/api/expenses	List / create expenses (scoped to user)	Required
PUT/DELETE	/api/expenses/:id	Update / delete an expense owned by the user	Required
GET/POST	/api/budgets	List / create budgets	Required
PUT/DELETE	/api/budgets/:id	Update / delete budget	Required
GET/POST	/api/savings	List / create savings goals	Required
PUT/DELETE	/api/savings/:id	Update / delete savings goal	Required
POST	/api/savings/:id/deposit	Deposit to a savings goal	Required
GET/POST	/api/investments	List / create investments	Required
PUT/DELETE	/api/investments/:id	Update / delete investment	Required
GET/POST	/api/categories	List / create categories	Required
PUT/DELETE	/api/categories/:id	Update / delete category	Required
GET	/api/transactions	List transactions (filters supported)	Required
GET	/api/transactions/export/csv	Export transactions as CSV	Required
GET	/api/transactions/export/pdf	Export transactions as PDF	Required
GET	/api/notifications	List notifications for user	Required
DELETE	/api/notifications/:id	Delete a notification	Required
PUT	/api/notifications/:id/read	Mark notification read	Required
GET	/api/dashboard/stats	Dashboard stats (guest-safe payload if unauthenticated)	Public

Method	Endpoint	Description	Auth
GET/PUT	/api/profile	Get / update profile	Required
PUT	/api/profile/change-password	Change user password	Required
GET/DELETE/PUT	/api/admin/*	Admin reports, list/delete users, suspend users	Admin

Table 5.1: FinPilot REST API reference.

CHAPTER 6

Security

FinPilot implements the following security mechanisms, each drawn from the actual implementation:

- **JWT authentication.** Stateless sessions using signed JSON Web Tokens that carry the user `id` and `role` and expire in 7 days.
- **bcrypt password hashing.** Passwords are hashed with `bcryptjs` in a pre-save hook and never stored in plain text; a `comparePassword()` helper validates logins.
- **Protected backend routes.** The `protect` middleware validates the Bearer token on every private route and populates `req.user`.
- **Role-based access control.** The `authorize(...roles)` middleware guards admin-only endpoints, returning `403 Forbidden` for insufficient roles.
- **User-specific data access.** Every controller query filters by `req.user.id`, so users can only read or modify their own records.
- **CORS restriction.** Both the HTTP API and the Socket.io channel restrict origins to the configured frontend URL.

CHAPTER 7

Deployment

FinPilot is deployed as a three-part cloud application:

Component	Platform
Frontend	Vercel (SPA rewrite rules configured)
Backend	Render (<code>render.yaml</code> blueprint; env vars <code>MONGO_URI</code> , <code>JWT_SECRET</code> , <code>FRONTEND_URL</code> , <code>PORT</code>)
Database	MongoDB Atlas (cloud-hosted, in use since Level 1)

Table 7.1: Deployment targets for each application layer.

Live URLs

- Frontend: <https://finpilot-oiek-q4zxkmfqt-sushil-bhattas-projects.vercel.app/>
- Backend API: <https://finplot-rhir.onrender.com>

CHAPTER 8

Final Project Summary

8.1 What Was Built

FinPilot is a complete full-stack personal finance and investment dashboard. Starting from a verified development environment and a simple Income/Expense REST API, the project grew into a React + TypeScript frontend, a JWT/bcrypt authentication system with role-based access control, a validated and indexed MongoDB data layer, and a full suite of finance modules — dashboard, income, expenses, budgets, savings goals, investments, transactions (with CSV/PDF export), categories, analytics, profile, and an admin panel — topped with real-time Socket.io notifications.

8.2 Technologies Learned and Applied

React 19, TypeScript, Vite, React Router, Axios, and Recharts on the frontend; Node.js, Express, JWT, bcryptjs, Socket.io, and PDFKit on the backend; and MongoDB with Mongoose for persistence — deployed across Vercel, Render, and MongoDB Atlas.

8.3 Full-Stack Concepts Demonstrated

- RESTful API design with full CRUD and centralized error handling.
- Component-based, type-safe frontend architecture with global state and typed API clients.
- Secure authentication, password hashing, and role-based authorization.
- Relational-style data modeling in MongoDB with validation, references, and indexing, scoped per user.
- Real-time client-server communication over WebSockets with private per-user rooms.
- Cloud deployment of a three-tier application.

8.4 Deployment Status

The application is deployed and accessible: the frontend on Vercel, the backend on Render, and the database on MongoDB Atlas.

8.5 Overall Outcome

All seven internship tasks across Levels 1–3 were completed, resulting in a cohesive, deployed full-stack application. FinPilot demonstrates end-to-end ownership of a modern web product — from environment setup and API design to authentication, data modeling, a polished UI, real-time features, and cloud deployment.